3D WEBSOFT INC. FOUNDATION

FUTURE TECH SCHOOL PROGRAM

Faculty Name :

Course Duration : 5 Months

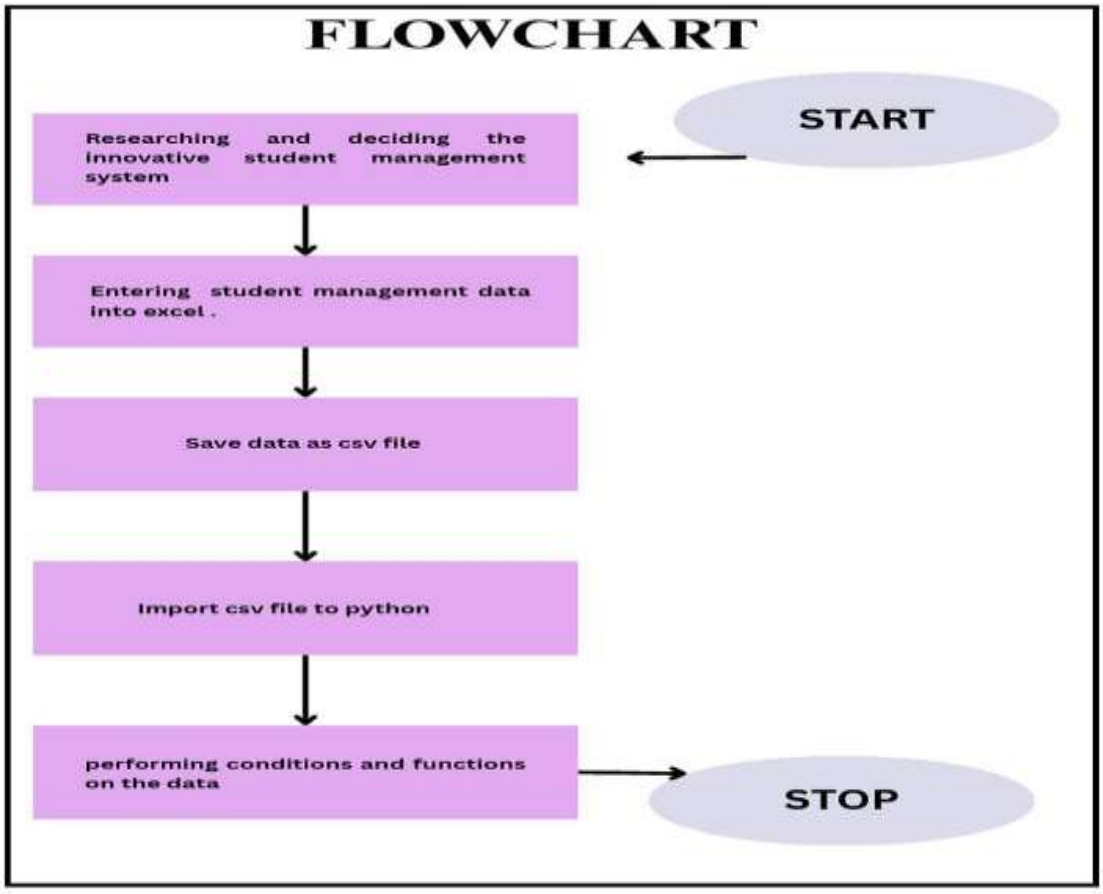
Classes : Offline & Online

Daily Teaching : 2 Hours

Total Classes : 100

Level 4 – AI Professional Foundation(11th–12th Standard)

Class 24 – Python Mini Project







1. Learning Objectives

By the end of this class, students will be able to:

- Understand what a Python mini project is.
- Convert a real-world problem into a Python application.
- Design a simple application using functions.

- Use variables, lists, dictionaries and loops.
- Use conditional statements.
- Use *args / **kwargs, lambda, map(), filter(), modules and exception handling where appropriate.
- Read and write data using CSV/JSON files.
- Create a menu-driven application.
- Test and debug a complete Python program.
- Understand how small Python applications are developed.

2. What is a Python Mini Project?

A **Python mini project** is a small application created using Python to solve a practical, real-world problem.

Instead of learning Python concepts separately, a project combines them into one application.

Example

A student database application can use:

Variables

↓

Lists / Dictionaries

↓

Functions

↓

Conditions & Loops

3DWEBSOFT FOUNDATION

↓

Exception Handling

↓

CSV / JSON

↓

Complete Python Application

Real-world Python applications

Application	Purpose
Student Management System	Store student information
Expense Tracker	Track daily expenses
Library Management	Manage books
Quiz Application	Conduct quizzes
Attendance System	Record attendance
To-Do List	Manage tasks
Result Analyzer	Analyze marks

3. Project Development Cycle

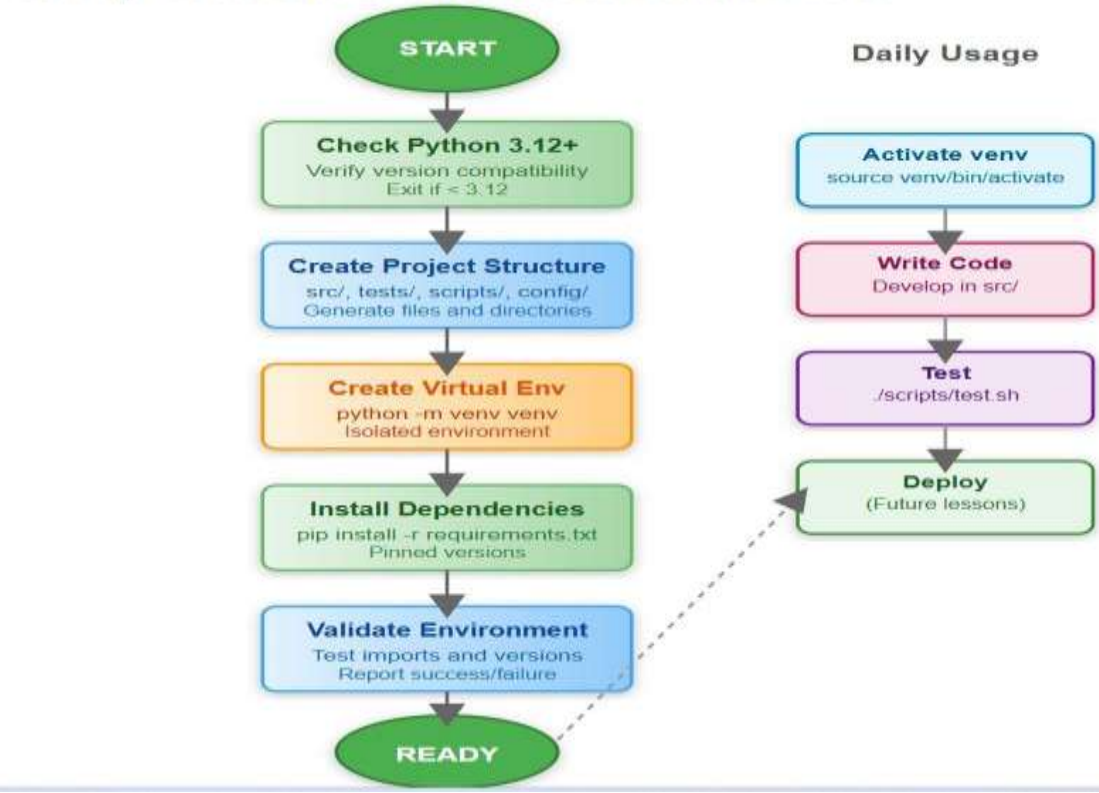
A good project is not written randomly. We follow a development process.

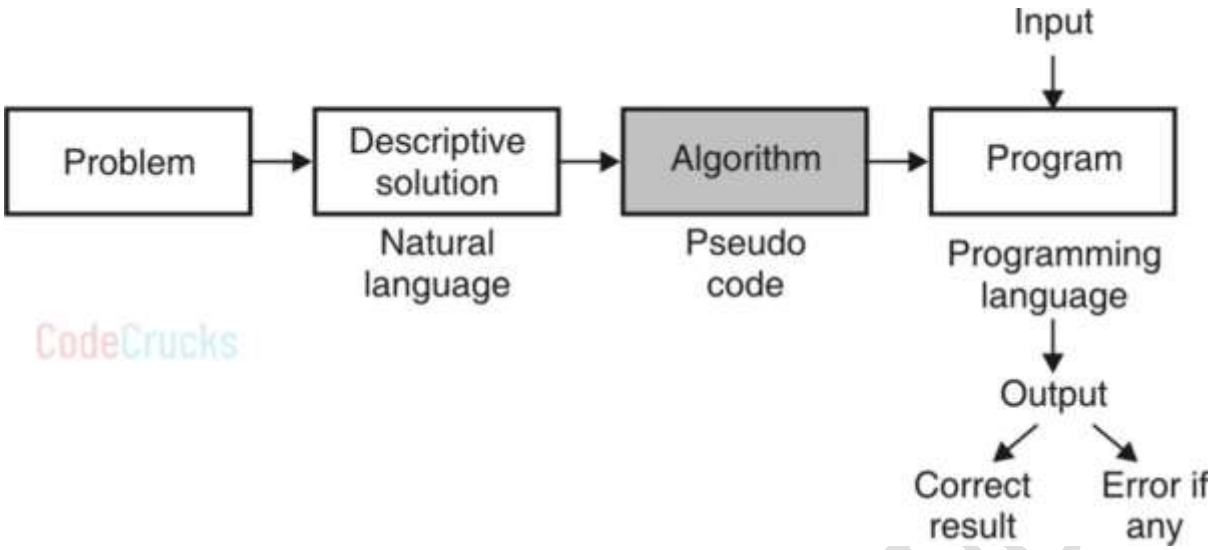
3DWEBSOFT FOUNDATION

Software Development Lifecycle



Setup Workflow and Execution Flow





CodeCrucks

Problem Identification

↓

Requirement Analysis

↓

Planning

↓

Program Design

↓

Coding

↓

Testing

↓

Debugging

↓

Final Application

3DWEBSOFT FOUNDATION

Step 1 – Identify the Problem

Ask:

What problem are we trying to solve?

Example:

Problem: A teacher wants to store and manage student marks digitally.

4. Mini Project: Student Performance Management System

For this class, we will build a practical **Student Performance Management System**.

Application Features

Student Performance Management System

```
|
 ├── Add Student
 ├── View Students
 ├── Search Student
 ├── Calculate Percentage
 ├── Display Grade
 ├── Find Top Student
 ├── Delete Student
 ├── Save Data
 └── Exit
```

3DWEBSOFT FOUNDATION

5. Project Requirements

Each student record will contain:

- Student ID
- Student Name
- Python Marks
- AI Marks
- Cybersecurity Marks

Example:

ID : 101

Name : Aisha

Python : 85

AI : 90

Cyber : 88

The application calculates:

Total Marks

Python + AI + Cybersecurity

Percentage

Total / 3

Grade

90–100 → A+

80–89 → A

70–79 → B

60–69 → C

3DWEBSOFT FOUNDATION

50–59 → D

Below 50 → F

6. Data Structure

We can represent one student using a dictionary.

```
student = {
    "id": 101,
    "name": "Aisha",
    "python": 85,
    "ai": 90,
    "cyber": 88
}
```

Multiple students can be stored in a list.

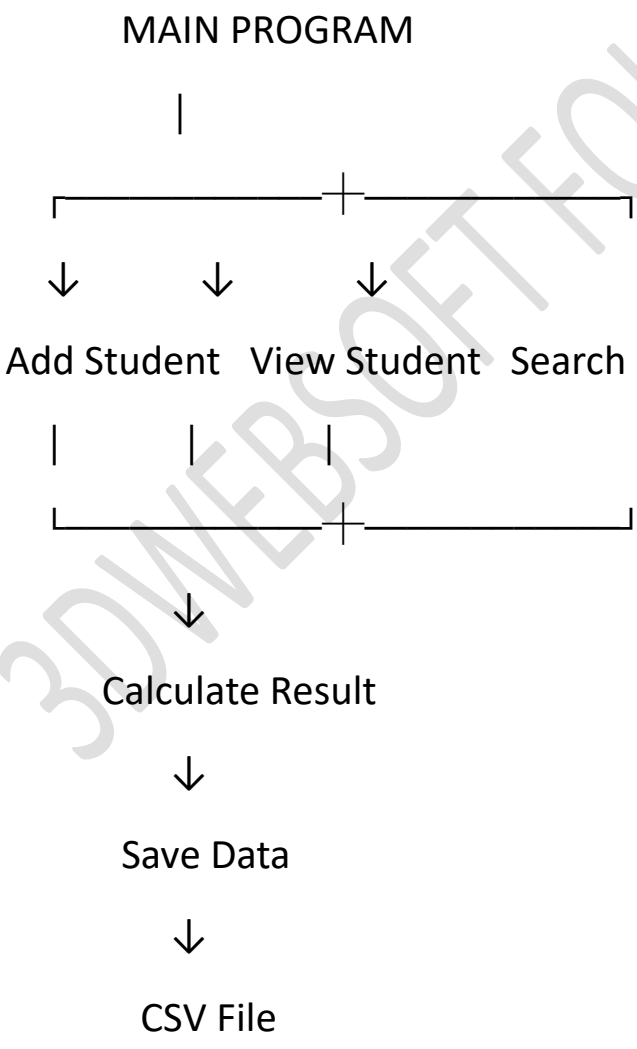
```
students = [
    {
        "id": 101,
        "name": "Aisha",
        "python": 85,
        "ai": 90,
        "cyber": 88
    },
    {
        "id": 102,
```

3DWEBSOFT FOUNDATION

```
    "name": "Rahul",
    "python": 78,
    "ai": 82,
    "cyber": 80
  }
]
```

7. Project Architecture

Our application will follow this structure:



3DWEBSOFT FOUNDATION

8. Creating the Main Menu

The menu is the main control center of our application.

```
while True:
    print("\n===== STUDENT PERFORMANCE SYSTEM =====")
    print("1. Add Student")
    print("2. View Students")
    print("3. Search Student")
    print("4. Exit")
    choice = input("Enter your choice: ")
    if choice == "1":
        print("Add Student")
    elif choice == "2":
        print("View Students")
    elif choice == "3":
        print("Search Student")
    elif choice == "4":
        print("Thank you!")
        break
    else:
        print("Invalid choice!")
```

Sample Output

```
===== STUDENT PERFORMANCE SYSTEM =====
```

3DWEBSOFT FOUNDATION

1. Add Student

2. View Students

3. Search Student

4. Exit

Enter your choice: 1

Add Student

9. Creating an Add Student Function

Instead of putting everything inside the main program, we use functions.

```
students = []

def add_student():
    student_id = input("Enter Student ID: ")
    name = input("Enter Student Name: ")
    python_marks = int(input("Enter Python marks: "))
    ai_marks = int(input("Enter AI marks: "))
    cyber_marks = int(input("Enter Cybersecurity marks: "))
    student = {
        "id": student_id,
        "name": name,
        "python": python_marks,
        "ai": ai_marks,
        "cyber": cyber_marks
```

3DWEBSOFT FOUNDATION

```
    }
    students.append(student)
    print("Student added successfully!")
```

10. View Students

```
def view_students():
    if not students:
        print("No students found.")
        return
    for student in students:
        print("\nStudent ID:", student["id"])
        print("Name:", student["name"])
        print("Python:", student["python"])
        print("AI:", student["ai"])
        print("Cybersecurity:", student["cyber"])
```

Output

Student ID: 101

Name: Aisha

Python: 85

AI: 90

Cybersecurity: 88

Student ID: 102

Name: Rahul

3JWEBSOFT FOUNDATION

Python: 78

AI: 82

Cybersecurity: 80

11. Calculate Percentage

We can create a reusable function.

```
def calculate_percentage(student):
    total = (
        student["python"]
        + student["ai"]
        + student["cyber"]
    )
    percentage = total / 3
    return percentage
```

Example:

```
student = {
    "python": 85,
    "ai": 90,
    "cyber": 88
}
percentage = calculate_percentage(student)
print("Percentage:", percentage)
```

Output:

3JWEBSOFT FOUNDATION

Percentage: 87.66666666666667

We can format it:

print(f"Percentage: {percentage:.2f}%")

Output:

Percentage: 87.67%

12. Grade Calculation

```
def calculate_grade(percentage):
    if percentage >= 90:
        return "A+"
    elif percentage >= 80:
        return "A"
    elif percentage >= 70:
        return "B"
    elif percentage >= 60:
        return "C"
    elif percentage >= 50:
        return "D"
    else:
        return "F"
```

Example:

```
percentage = 87.67
grade = calculate_grade(percentage)
```

3DWEBSOFT FOUNDATION

```
print("Grade:", grade)
```

Output:

Grade: A

13. Search Student

```
def search_student():
    student_id = input("Enter Student ID: ")
    for student in students:
        if student["id"] == student_id:
            print("\nStudent Found!")
            print("Name:", student["name"])
            print("Python:", student["python"])
            print("AI:", student["ai"])
            print("Cyber:", student["cyber"])
            percentage = calculate_percentage(student)
            print(f"Percentage: {percentage:.2f}%")
            print("Grade:", calculate_grade(percentage))
            return
    print("Student not found.")
```

14. Finding the Top Student

Python's max() function can help us find the student with the highest marks.

3DWEBSOFT FOUNDATION

```
def top_student():
    if not students:
        print("No students available.")
        return
    top = max(
        students,
        key=calculate_percentage
    )
    percentage = calculate_percentage(top)
    print("\n===== TOP STUDENT =====")
    print("Name:", top["name"])
    print(f"Percentage: {percentage:.2f}%")
    print("Grade:", calculate_grade(percentage))
```

This demonstrates an important Python idea:

List of students

↓

Calculate percentage

↓

Compare percentages

↓

Find maximum

↓

Top student

3DWEBSOFT FOUNDATION

15. Exception Handling

Users may enter incorrect information.

For example:

```
marks = int(input("Enter marks: "))
```

If the user enters:

```
abc
```

Python generates a ValueError.

We can handle it.

```
try:
    marks = int(input("Enter marks: "))
except ValueError:
    print("Please enter numbers only.")
```

Better validation

```
def get_marks(subject):
    while True:
        try:
            marks = int(input(f"Enter {subject} marks: "))
            if 0 <= marks <= 100:
                return marks
            else:
                print("Marks must be between 0 and 100.")
        except ValueError:
            print("Please enter a valid number.")
```

3DWEBSOFT FOUNDATION

Now:

```
python_marks = get_marks("Python")
ai_marks = get_marks("AI")
cyber_marks = get_marks("Cybersecurity")
```

16. Saving Student Data to CSV

Our program should not lose data when it closes.

We can use the CSV module.

```
import csv
```

Save Function

```
def save_students():
    with open("students.csv", "w", newline="") as file:
        fieldnames = [
            "id",
            "name",
            "python",
            "ai",
            "cyber"
        ]
        writer = csv.DictWriter(
            file,
            fieldnames=fieldnames
        )
```

3DWEBSOFT FOUNDATION

```
        writer.writeheader()
        writer.writerows(students)
    print("Data saved successfully!")
```

17. Loading Data

When the program starts, we can load existing students.

```
def load_students():
    try:
        with open("students.csv", "r") as file:
            reader = csv.DictReader(file)
            for row in reader:
                row["python"] = int(row["python"])
                row["ai"] = int(row["ai"])
                row["cyber"] = int(row["cyber"])
                students.append(row)
    except FileNotFoundError:
        pass
```

This combines concepts from our previous classes:

CSV

+

Functions

+

Exception Handling

3DWEBSOFT FOUNDATION

+

Lists

+

Dictionaries

=

Practical Application

18. Complete Mini Project

Here is the complete classroom version.

```
import csv
students = []
def get_marks(subject):
    while True:
        try:
            marks = int(input(f"Enter {subject} marks: "))
            if 0 <= marks <= 100:
                return marks
            print("Marks must be between 0 and 100.")
        except ValueError:
            print("Please enter a valid number.")
def calculate_percentage(student):
    total = (
        student["python"]
```

3DWEBSOFT FOUNDATION

```
        + student["ai"]
        + student["cyber"]
    )
    return total / 3
def calculate_grade(percentage):
    if percentage >= 90:
        return "A+"
    elif percentage >= 80:
        return "A"
    elif percentage >= 70:
        return "B"
    elif percentage >= 60:
        return "C"
    elif percentage >= 50:
        return "D"
    else:
        return "F"
def add_student():
    student_id = input("Enter Student ID: ")
    name = input("Enter Student Name: ")
    python_marks = get_marks("Python")
    ai_marks = get_marks("AI")
    cyber_marks = get_marks("Cybersecurity")
```

SUWEBSOFT FOUNDATION

```
    student = {
        "id": student_id,
        "name": name,
        "python": python_marks,
        "ai": ai_marks,
        "cyber": cyber_marks
    }
    students.append(student)
    print("Student added successfully!")
def view_students():
    if not students:
        print("No students found.")
        return
    for student in students:
        percentage = calculate_percentage(student)
        grade = calculate_grade(percentage)
        print("\n----------------------")
        print("ID:", student["id"])
        print("Name:", student["name"])
        print("Python:", student["python"])
        print("AI:", student["ai"])
        print("Cybersecurity:", student["cyber"])
        print(f"Percentage: {percentage:.2f}%")
```

3DWEBSOFT FOUNDATION

```
        print("Grade:", grade)
def search_student():
    student_id = input("Enter Student ID: ")
    for student in students:
        if student["id"] == student_id:
            percentage = calculate_percentage(student)
            grade = calculate_grade(percentage)
            print("\nStudent Found!")
            print("Name:", student["name"])
            print(f"Percentage: {percentage:.2f}%")
            print("Grade:", grade)
            return
    print("Student not found.")
def top_student():
    if not students:
        print("No students available.")
        return
    top = max(
        students,
        key=calculate_percentage
    )
    percentage = calculate_percentage(top)
    print("\n===== TOP STUDENT =====")
```

SUWEBSOFT FOUNDATION

```
    print("Name:", top["name"])
    print(f"Percentage: {percentage:.2f}%")
    print("Grade:", calculate_grade(percentage))
def save_students():
    with open("students.csv", "w", newline="") as file:
        fieldnames = [
            "id",
            "name",
            "python",
            "ai",
            "cyber"
        ]
        writer = csv.DictWriter(
            file,
            fieldnames=fieldnames
        )
        writer.writeheader()
        writer.writerows(students)
    print("Data saved successfully!")
while True:
    print("\n===== STUDENT PERFORMANCE SYSTEM =====")
    print("1. Add Student")
    print("2. View Students")
```

3DWEBSOFT FOUNDATION

```
    print("3. Search Student")
    print("4. Top Student")
    print("5. Save Data")
    print("6. Exit")
    choice = input("Enter your choice: ")
    if choice == "1":
        add_student()
    elif choice == "2":
        view_students()
    elif choice == "3":
        search_student()
    elif choice == "4":
        top_student()
    elif choice == "5":
        save_students()
    elif choice == "6":
        print("Thank you for using the application!")
        break
    else:
        print("Invalid choice. Try again.")
```

19. Sample Project Output

```
===== STUDENT PERFORMANCE SYSTEM =====
```

3DWEBSOFT FOUNDATION

1. Add Student

2. View Students

3. Search Student

4. Top Student

5. Save Data

6. Exit

Enter your choice: 1

Enter Student ID: 101

Enter Student Name: Aisha

Enter Python marks: 85

Enter AI marks: 90

Enter Cybersecurity marks: 88

Student added successfully!

Then:

Enter your choice: 2

ID: 101

Name: Aisha

Python: 85

AI: 90

Cybersecurity: 88

3DWEBSOFT FOUNDATION

Percentage: 87.67%

Grade: A

20. How Python Concepts Were Used

Python Concept	Used in Project
Variables	Student information
Strings	Student names
Integers	Marks
Lists	Multiple students
Dictionaries	Student records
Functions	Application features
if-elif-else	Grade calculation
for loop	Searching/displaying
while loop	Menu
max()	Top student
Exception Handling	Invalid marks
CSV	Data storage
Modules	csv
File Handling	Saving data

This is the main purpose of a mini project: **combining previously learned concepts into one useful application.**

3DWEBSOFT FOUNDATION

21. Interactive Classroom Activity

Activity 1 – Predict the Output

Ask students:

```
marks = [80, 90, 70]


average = sum(marks) / len(marks)


print(average)
```

Answer:

80.0

Activity 2 – Find the Error

```
marks = int(input("Enter marks: "))


if marks > 100:
    print("Invalid")
```

Ask students:

What happens if the user enters hello?

Expected answer:

ValueError

22. Student Challenge

Modify the project to add:

3DWEBSOFT FOUNDATION

Challenge 1

Add a **Delete Student** feature.

Challenge 2

Add an **Update Student** feature.

Challenge 3

Add a **Total Marks** display.

Challenge 4

Find students who scored more than 80%.

Hint:

```
for student in students:
    percentage = calculate_percentage(student)
    if percentage > 80:
        print(student["name"])
```

Challenge 5

Use filter() to find students above 80%.

```
high_performers = list(
    filter(
        lambda s: calculate_percentage(s) > 80,
        students
    )
)
```

This connects the project with **Class 12 – map(), filter() and reduce()**.

3DWEBSOFT FOUNDATION

23. Mini Project Extension

Students can make the project more advanced:

Student Performance System

```
        |
        ├── Student Registration
        ├── Marks Entry
        ├── Result Calculation
        ├── Grade Calculation
        ├── Search
        ├── Update
        ├── Delete
        ├── Top Performer
        ├── CSV Storage
        ├── JSON Backup
        └── Reports
```

Advanced version

Students can later add:

- Login system
- Teacher login
- Student login
- Password protection
- JSON storage

3DWEBSOFT FOUNDATION

- Pandas data analysis
- Matplotlib charts
- GUI using Tkinter
- Database using SQLite

24. Common Mistakes

Mistake 1 – Forgetting indentation

Incorrect:

```
if marks >= 50:
print("Pass")
```

Correct:

```
if marks >= 50:
    print("Pass")
```

Mistake 2 – Comparing different data types

```
student_id = input("Enter ID: ")


if student_id == 101:
    print("Found")
```

Here student_id is a string.

Better:

```
if student_id == "101":
    print("Found")
```

Mistake 3 – Forgetting to save data

3JWEBSOFT FOUNDATION

If the program only uses:

students = []

the data disappears when the program closes.

Use CSV/JSON for permanent storage.

25. Testing the Application

Students should test different situations.

Test	Expected Result
Valid marks	Student added
Marks = 100	Accepted
Marks = 0	Accepted
Marks = 105	Rejected
Marks = -5	Rejected
Text instead of marks	Error handled
Existing student search	Student displayed
Unknown student search	Not found
Empty student list	Proper message

26. Project Presentation Activity

Each student/team should present:

1. Project title

9JWEBSOFT FOUNDATION

2. Problem statement
3. Objectives
4. Features
5. Python concepts used
6. Program demonstration
7. Output
8. Challenges faced
9. How errors were fixed
10. Future improvements

Presentation Flow

Problem

↓

Solution

↓

Python Concepts

↓

Program

↓

Output

↓

Testing

↓

Future Improvements

3DWEBSOFT FOUNDATION

27. Practical Assignment

Build Your Own Python Application

Students can choose **one**:

Option A: Student Management System
Option B: Expense Tracker
Option C: Library Management System
Option D: Quiz Application
Option E: Attendance Management System
Option F: To-Do List Application

Minimum Requirements

The project must contain:

- At least 4 functions
- Menu-driven interface
- Lists/dictionaries
- Conditions
- Loops
- Exception handling
- File storage
- Search functionality
- At least one calculation
- Proper output messages

3DWEBSOFT FOUNDATION

28. Viva Questions

Q1. What is a mini project?

A small practical application developed to solve a particular problem.

Q2. Why are functions useful?

Functions divide a large program into smaller reusable parts.

Q3. Why use dictionaries?

They allow related information to be stored using key-value pairs.

Q4. Why use CSV?

CSV provides a simple way to store tabular data.

Q5. Why use exception handling?

It prevents unexpected errors from crashing the application.

Q6. What is a menu-driven program?

A program where users select operations from a menu.

Q7. Why do we test a project?

To find and fix errors and verify that the application works correctly.

3DWEBSOFT FOUNDATION

29. MCQs

1. Which data structure is useful for storing one student's details?

A. Dictionary
B. Integer
C. Boolean
D. Operator

Answer: A

2. Which keyword is used to define a function?

A. function
B. define

C. def
D. fun

Answer: C

3. Which module is used for CSV files?

A. csv
B. file
C. data
D. table

Answer: A

4. Which block handles an exception?

A. try-except
B. if-else
C. for
D. def

Answer: A

5. Which function can find the largest value?

A. top()
B. maximum()
C. max()
D. largest()

Answer: C

3D WEBSOFT FOUNDATION

30. Quick Revision

Python Mini Project

↓

Identify Problem

↓

Plan Features

↓

Choose Data Structures

↓

Create Functions

↓

Write Main Menu

↓

Add Validation

↓

Handle Exceptions

↓

Store Data

↓

Test

↓

Debug

↓

Final Application

Final Takeaway

3DWEBSOFT FOUNDATION

A Python programmer should not only know individual concepts such as **functions, lists, dictionaries, files and exception handling**. The real skill is knowing **how to combine those concepts to solve a real-world problem**.

3DWEBSOFT FOUNDATION